6.0 Architectural Design Document

6.1 Introduction

This document presents the architecture for Poker Face 2, an evolution of the original Poker Face project. Poker Face 2 expands the system from a local single-player simulation into a modular, web-enabled platform that supports human players, AI opponents, and future multiplayer interactions.

The system allows users to play No-Limit Texas Hold'em Poker through a web interface while interacting with adaptive AI agents for real-time gameplay, remote connections, and scalable deployment.

6.1.1 System Objectives

The objective of Poker Face 2 is to answer an extended version of the original research question:

Can an AI agent that continuously learns from human behavioral patterns in poker—without access to physical tells—adapt across sessions and outperform human players in both short-term and long-term play?

Poker Face 2 introduces several new goals:

- Enable persistent opponent modeling across sessions
- Support real-time web-based gameplay
- Allow multiple AI strategies and models to coexist
- Provide a foundation for multiplayer and multi-agent environments
- Collect structured gameplay data for future research and model improvement

6.1.2 Hardware, Software, and Human Interfaces

6.1.2.1 Hardware Interface

6.1.2.1.1 Client Devices

Users shall access Poker Face 2 through:

- Desktop computers
- Laptops

6.1.2.1.2 Server Infrastructure

The system shall run on a backend server, which may be:

- Local development machine
- Cloud-hosted instance (e.g., AWS, GCP, or similar)

6.1.2.2 Software Interface

The sub-sections of this point list the programs used

6.1.2.2.1 Backend (Python)
Python 3.10+
Framework: FastAPI
AI/ML Libraries:

- PyTorch 2.0+
- NumPy

Data handling:

- JSON APIs

6.1.2.2.2 Frontend (Web Client)
React
TypeScript
WebSocket support for real-time updates
REST API integration for game actions

6.1.2.2.3 Communication Layer
REST APIs for standard requests (e.g., actions, game state)
WebSockets for real-time game updates

6.1.2.3 Human Interface

6.1.2.3.1 Input Methods
Mouse/trackpad

6.1.2.3.2 Web GUI
Game Controls

- Check Button
  - Advances turn without betting.
- Call Button
  - Matches the current bet if sufficient chips are available.
- Raise Button
  - Prompts user to input a raise amount greater than the current bet.
- Fold Button
  - Removes the player from the current hand.

6.2 Architectural Design
Poker Face 2 adopts a client-server architecture with modular AI services. The system is divided into four primary subsystems:

1. Frontend Client (UI Layer)
2. Backend Game Server (State Control)
3. AI Service (Computer Player)
4. Persistence Layer (Data Storage)

6.2.1 Major Software Components

6.2.1.1 Frontend Client Subsystem

The frontend provides the user interface via a web application. It renders:

- Cards, chips, and game state
- Player actions
- Real-time updates via WebSockets

It communicates exclusively with the backend via APIs.

6.2.1.2 Backend Game Server Subsystem

The backend serves as the authoritative game engine. Responsibilities include:

- Enforcing poker rules
- Managing game state
- Validating actions
- Handling turn order and phases
- Broadcasting updates to clients

6.2.1.3 AI Service Subsystem

The AI subsystem is modular and runs as a service. It:

- Converts game state into feature vectors
- Evaluates actions using neural networks or heuristics
- Maintains persistent opponent models
- Supports multiple AI strategies (future extensibility)

6.2.1.4 Persistence Subsystem

Unlike Poker Face 1, this version introduces optional persistence:

- Stores player statistics
- Saves AI models and learned parameters
- Maintains game history logs

6.2.2 Major Software Interactions

6.2.2.1 Frontend → Backend

User actions are sent via REST API calls.

6.2.2.2 Backend → Frontend

Game state updates are sent via WebSockets.

6.2.2.3 Backend → AI Service

The backend sends the current game state when it is the AI's turn.

6.2.2.4 AI Service → Backend

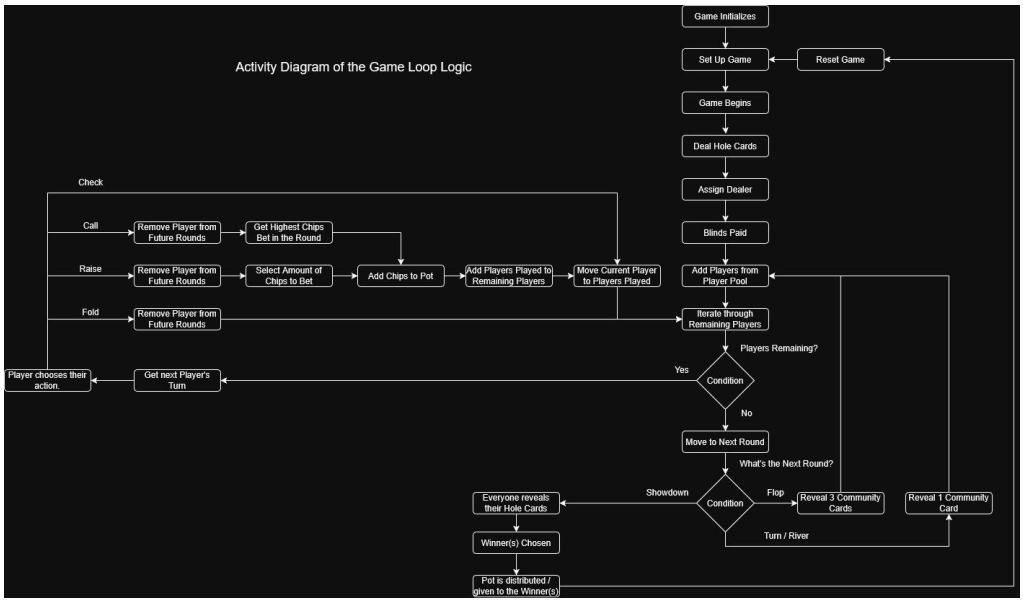
The AI returns a selected action.

6.2.2.5 Backend ↔ Persistence

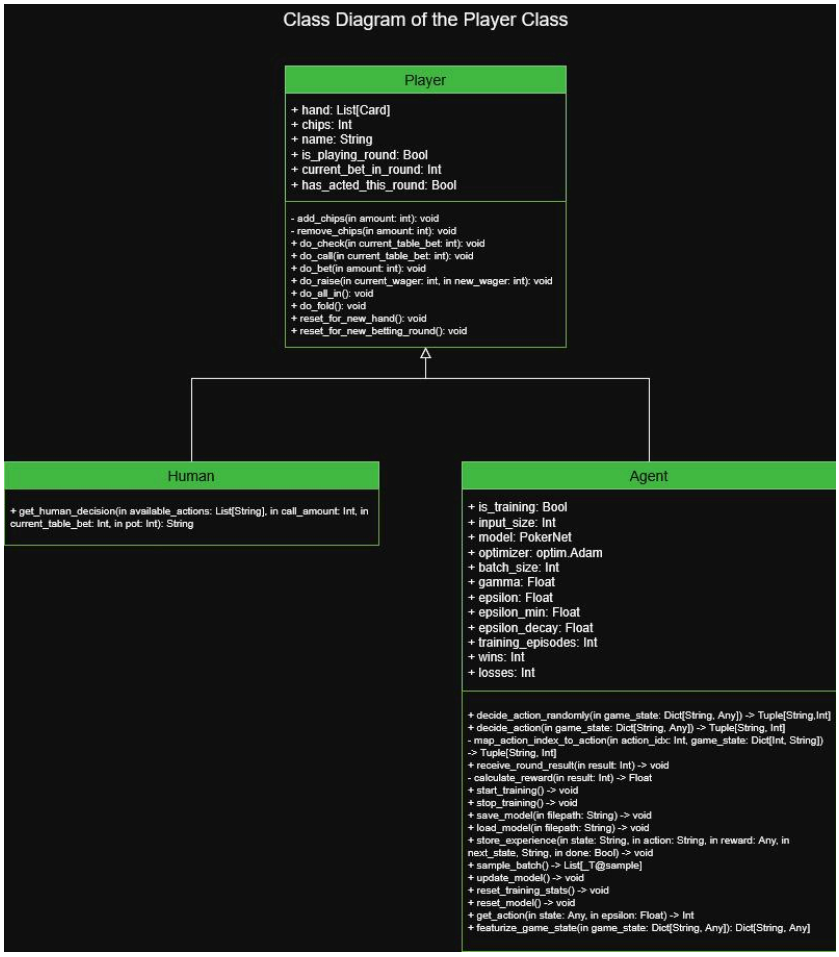
Game data and models are stored and retrieved as needed.

6.2.3 Architectural Design Diagrams Section

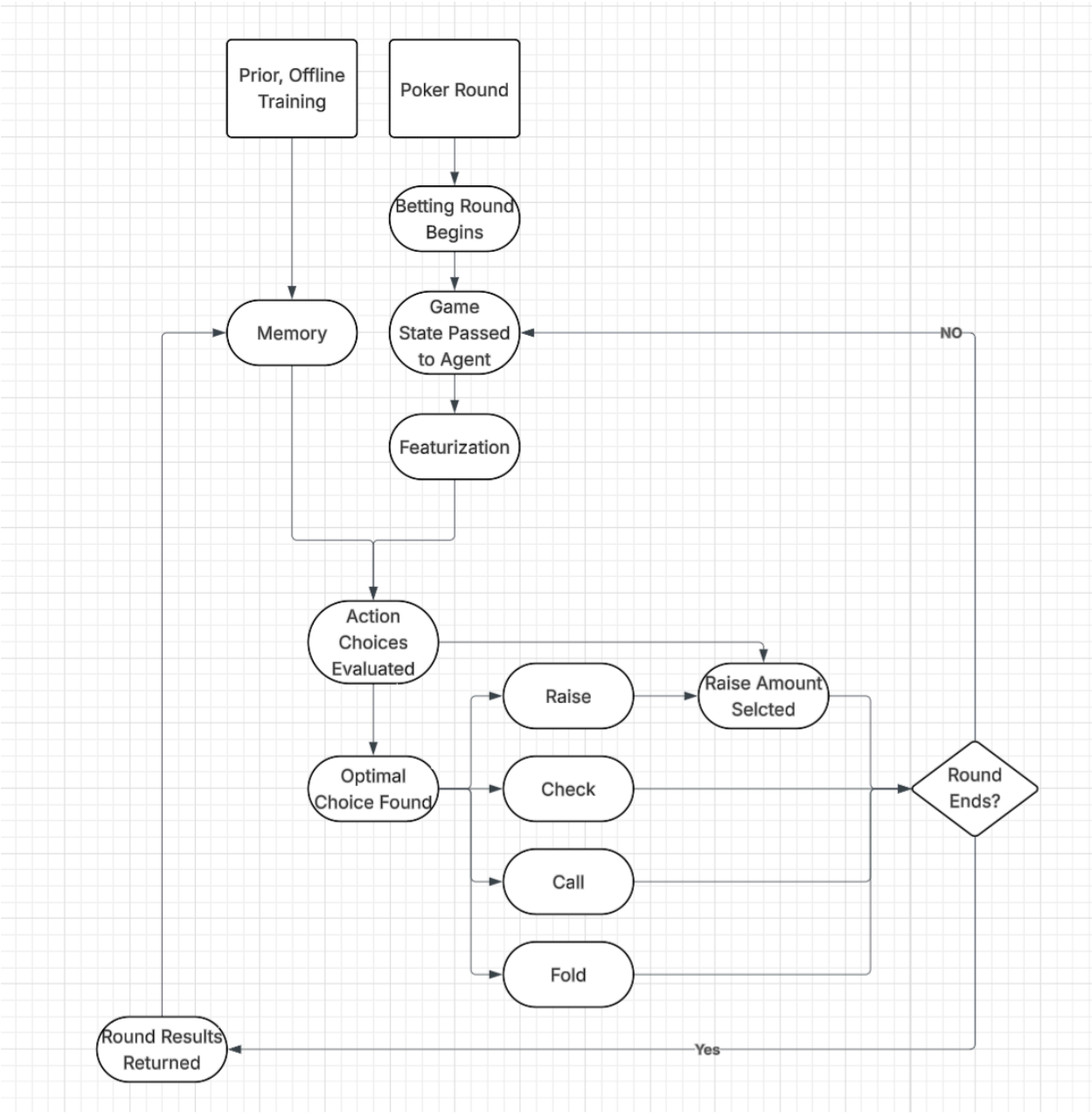
6.2.3.1 Game Loop Diagram



6.2.3.2 Player Class Diagram



6.2.3.3 Agent Class Diagrams



6.3 Detailed CSC and CSU Descriptions

Poker Face 2 consists of four CSCs:

1. Frontend (React)
2. Backend (Game Logic API)
3. AI Service
4. Persistence Layer

6.3.1 Detailed Class Descriptions

Card Class

- Represents a playing card.

Deck Class
- Manages deck creation, shuffling, and dealing.

Player Class
- Represents human or AI players, including chip count and actions.

GameState Class
- Central state container shared across backend and AI.

AgentModel Class
- Encapsulates neural network and opponent modeling logic.

APIController Class
- Handles HTTP/WebSocket communication between frontend and backend.

GameController Class
- Coordinates gameplay and orchestrates subsystem interaction.

6.3.2 Detailed Interface Descriptions

6.3.2.1 GUI ↔ Game Logic Interface

The GUI and Game Logic subsystems interact when the player acts, such as Fold, Call, Raise, or Check. The GUI sends the user's chosen action to the Game Logic, which validates it and returns an updated game state for display.

6.3.2.2 Game Logic ↔ AI Agent Interface

The Game Logic subsystem provides the AI Agent with a structured representation of the current game, allowing the model to analyze the state and determine an optimal action. The AI then returns its selected move, which the Game Logic applies to progress the hand.

6.3.2.3 Game Logic ↔ GUI Interface

After each state update, the Game Logic subsystem communicates back to the GUI to refresh the visual presentation. The updated information includes chip counts, pot size, community cards, and game status, ensuring the player always sees an accurate reflection of play.

6.3.3 Detailed Data Structures

6.3.3.1 Feature Vector Structure

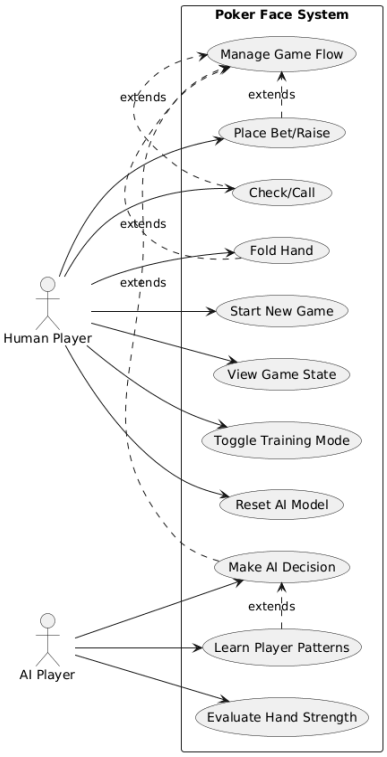
The Feature Vector structure encodes the current game state into a numerical format suitable for neural network processing. It includes values such as the player's hand strength, pot odds, opponent aggression index, phase of play, and relative chip advantage. This compact representation allows the AI Agent to evaluate strategic options efficiently during decision-making.

6.3.3.2 Action Object Structure

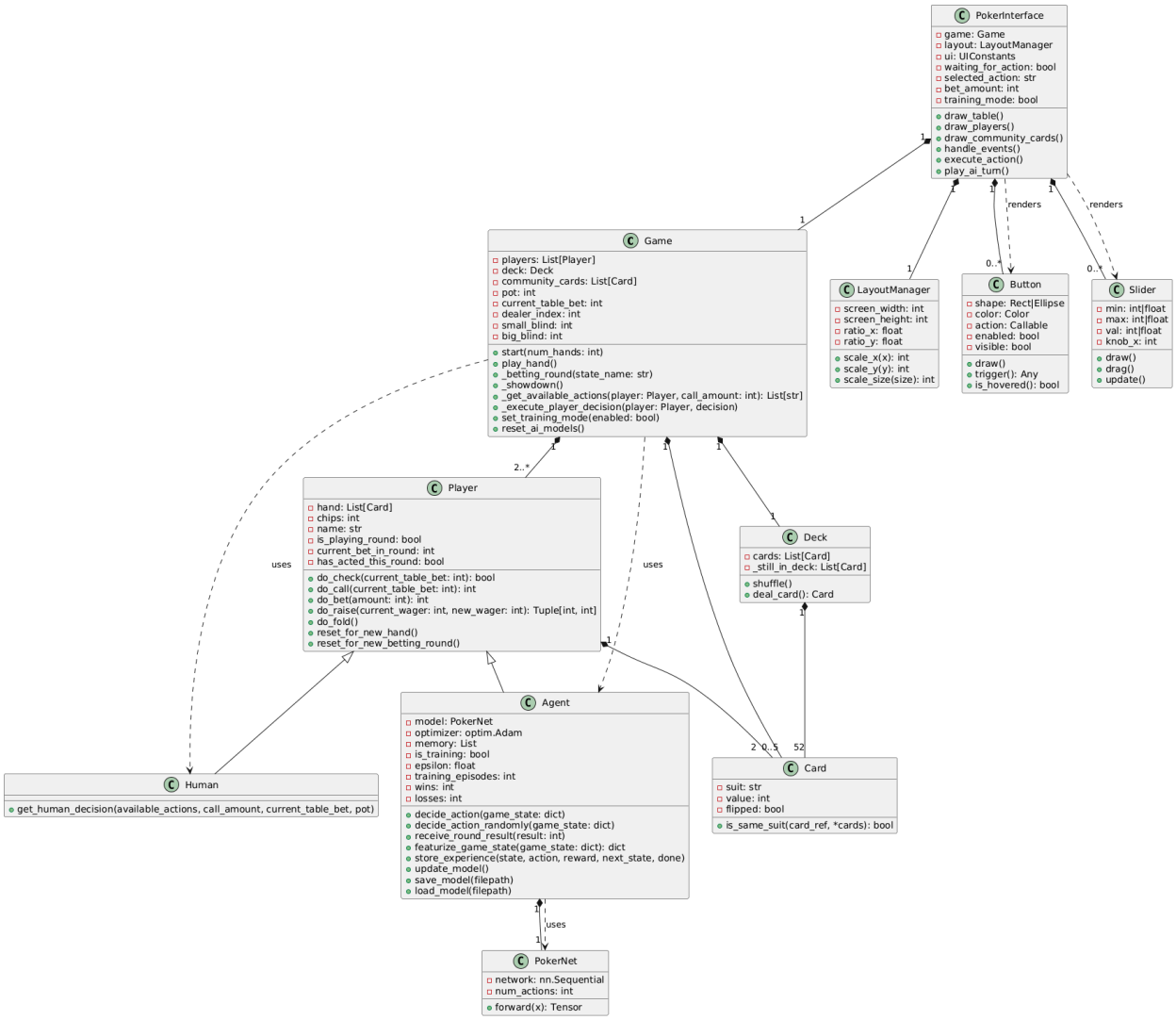
The Action Object structure encapsulates the details of a single move made by either the player or the AI. It records the type of action—such as Fold, Call, Raise, or Check—the amount of chips wagered if applicable, and the actor responsible for the move. This object enables consistent communication between the Game Logic, AI Agent, and GUI subsystems, ensuring that all actions are processed and displayed accurately.

6.3.4 Detailed Design Diagrams

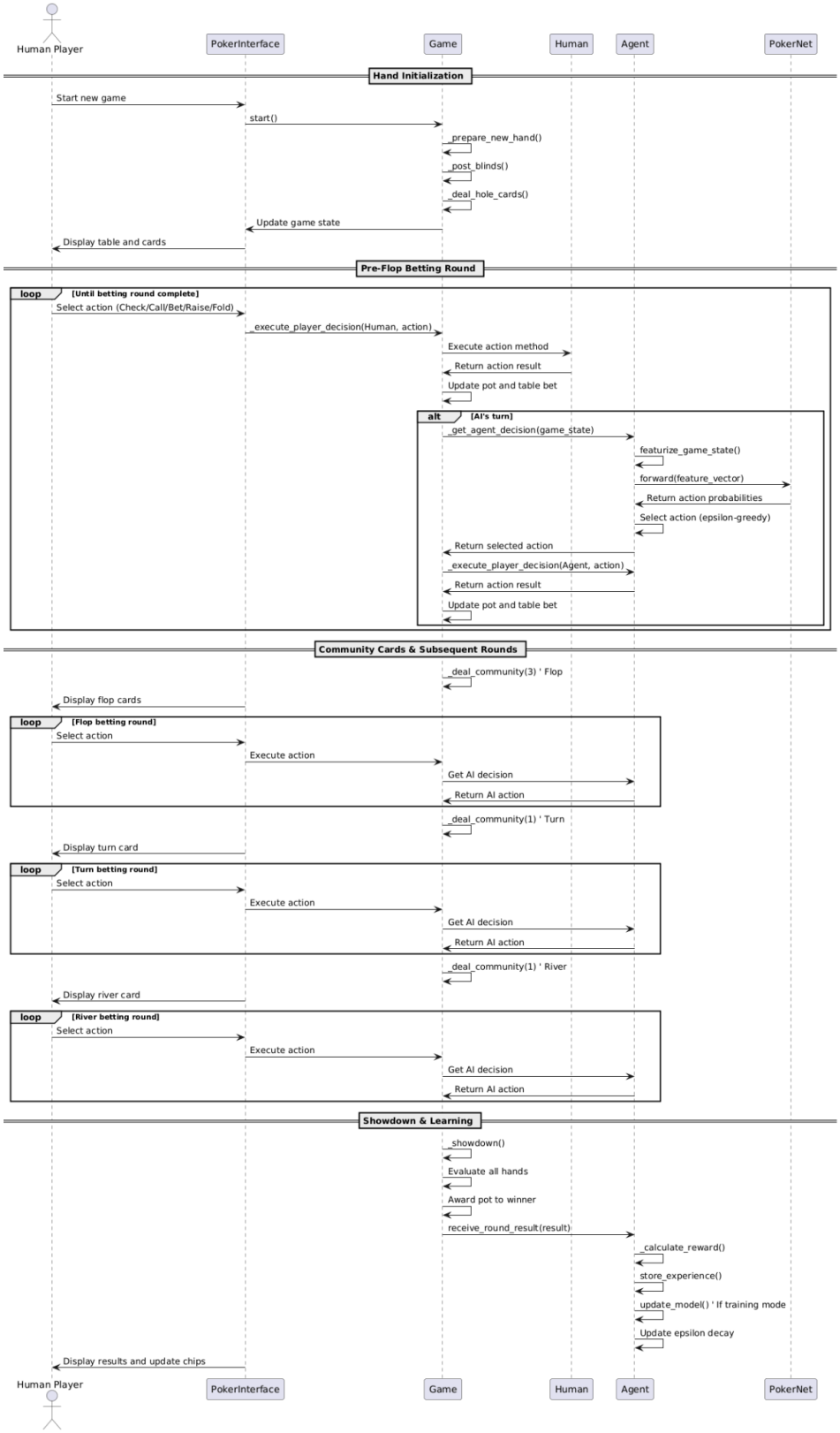
6.3.4.1 Use Case Diagram



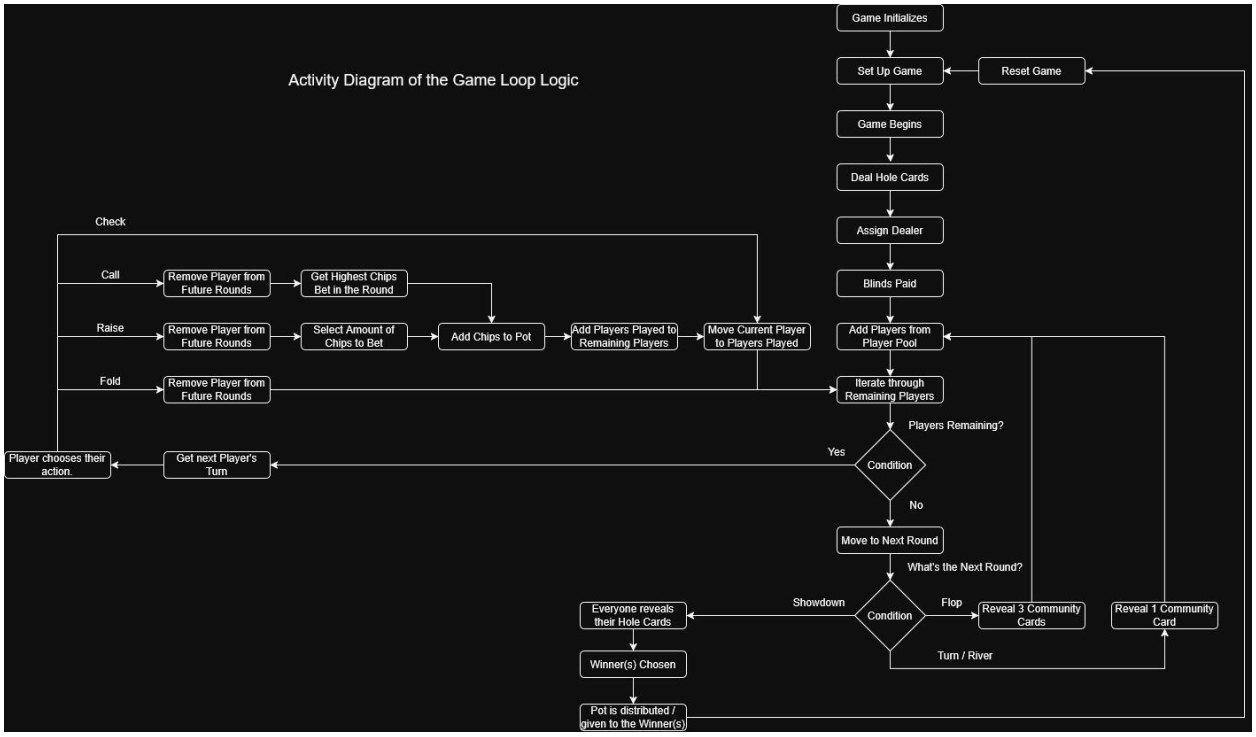
6.3.4.2 Class Diagram



6.3.4.3 Sequence Diagram



6.3.4.4 Activity Diagram



6.4 Database Design and Description

Pokerface doesn't implement any database; rather, all behavior and decisions are learned from the agent's gameplay experience.